



🇬🇧 English (Original)



Section 12 / 24

Go to Questions ▾

Arithmetic Instructions

The second type of basic instructions is Arithmetic Instructions. With Arithmetic Instructions, we can perform various mathematical computations on data stored in registers and memory addresses. These instructions are usually processed by the ALU in the CPU, among other instructions. We will split arithmetic instructions into two types: instructions that take only one operand (`Unary`), instructions that take two operands (`Binary`).

Unary Instructions

The following are the main Unary Arithmetic Instructions (we will assume that `rax` starts as `1` for each instruction):

Instruction	Description	Example
<code>inc</code>	Increment by 1	<code>inc rax</code> -> <code>rax++</code> or <code>rax += 1</code> -> <code>rax = 2</code>
<code>dec</code>	Decrement by 1	<code>dec rax</code> -> <code>rax--</code> or <code>rax -= 1</code> -> <code>rax = 0</code>

Let's practice these instructions by going back to our `fib.s` code. So far, we have initialized `rax` and `rbx` with our initial values `0` and `1` with the `mov` instruction. Instead of moving the immediate value of `1` to `bl`, let's move `0` to it, and then use `inc` to make it `1`:

```
nasm
global _start
section .text
_start:
    mov al, 0
    mov bl, 0
    inc bl
```

Remember, we used `al` instead of `rax` for efficiency. Now, let us assemble our code, and run it with `gdb`:

```
$ ./assembler.sh fib.s -g
...SNIP...

code:x86:64 ———
→ 0x401005 <_start+5>      mov    al,
0x0
registers ——— $rbx      : 0x0

...SNIP...

code:x86:64 ———
→ 0x40100a <_start+10>     inc
bl
registers ——— $rbx      : 0x1
```

As we can see, `rbx` started with the value `0`, and with `inc rbx`, it was incremented to `1`. The `dec` instruction is similar to `inc`, but decrements by `1` instead of incrementing.

This knowledge will become very handy later on.

Binary Instructions

Next, we have Binary Arithmetic Instructions, and the main ones are: We'll assume that both `rax` and `rbx` start as `1` for each instruction.

Instruction	Description	Example
<code>add</code>	Add both operands	<code>add rax, rbx</code> -> <code>rax = 1 + 1</code> -> <code>2</code>
<code>sub</code>	Subtract Source from Destination (i.e. <code>rax = rax - rbx</code>)	<code>sub rax, rbx</code> -> <code>rax = 1 - 1</code> -> <code>0</code>
<code>imul</code>	Multiply both operands	<code>imul rax, rbx</code> -> <code>rax = 1 * 1</code> -> <code>1</code>

Note that in all of the above instructions, the result is always stored in the destination operand, while the source operand is not affected.

Let's start by discussing the `add` instruction. Adding two numbers is the core step of calculating a Fibonacci Sequence, since the current Fibonacci number (F_n) is the sum of the two preceding it ($F_n = F_{n-1} + F_{n-2}$).

So, let's add `add rax, rbx` to the end of our `fib.s` code:

```
global _start

section .text
_start:
    mov al, 0
    mov bl, 0
    inc bl
    add rax, rbx
```

nasm

Now, let's assemble our code, and run it with `gdb`:

```
$ ./assembler.sh fib.s -g
gef> b _start
Breakpoint 1 at 0x401000
gef> r
...SNIP...
```

```
code:x86:64 -----
    0x401004 <_start+4>      inc    bl
→   0x401006 <_start+6>      add     rax,
rbx-----
registers -----$rax    : 0x1
$rbx    : 0x1
```

As we can see, after the instruction is processed, `rax` is equal to `0x1 + 0x0`, which is `0x1`. Using the same principle, if we had other Fibonacci numbers in `rax` and `rbx`, we'd get the new Fibonacci using `add`.

Both `sub` and `imul` are similar to `add`, as shown in the examples in the previous table. Try adding `sub` and `imul` to the above code, assemble it, and then run it `gdb` to see how they work.

Bitwise Instructions

Now, let's move to Bitwise Instructions, which are instructions that work on the bit level (we'll assume that `rax = 1` and `rbx = 2` for each instruction):

Instruction	Description	Example
<code>not</code>	Bitwise NOT (invert all bits, 0->1 and 1->0)	<code>not rax</code> -> NOT 00000001 -> 11111110
<code>and</code>	Bitwise AND (if both bits are 1 -> 1, if bits are different -> 0)	<code>and rax, rbx</code> -> 00000001 AND 00000010 -> 00000000
<code>or</code>	Bitwise OR (if either bit is 1 -> 1, if both are 0 -> 0)	<code>or rax, rbx</code> -> 00000001 OR 00000010 -> 00000011
<code>xor</code>	Bitwise XOR (if bits are the same -> 0, if bits are different -> 1)	<code>xor rax, rbx</code> -> 00000001 XOR 00000010 -> 00000011

These instructions may look confusing at first, but they are straightforward once we understand them. Each of these instructions makes the specified instruction on each bit of the value. For example, `not` will go to each bit and invert it, turning 0's to 1's and turning 1's to 0's. Try adding `not rax` to the end of our previous code, assemble it, and then run it with `gdb` to see how it works.

Likewise, both the `and` / `or` instructions work on each bit, and perform the `AND` / `OR` gate on each one, as shown in the examples above. Each of these instructions has its use cases in Assembly.

However, the instruction we will be using the most is `xor`. The `xor` instruction has various use cases, but since it zeros similar bits, we can use it to turn any value to 0 by `xor`ing a value with itself. We need to put, `using xor on any register with itself will turn it into 0`.

For example, if we want to turn the `rax` register to `0`, the most efficient way to do it is `xor rax, rax`, which will make `rax = 0`. This is simply because all bits of `rax` are similar, and so `xor` will turn all of them to `0`. Going back to our previous `fib.s` code, instead of moving `0` to both `rax` and `rbx`, we can use `xor` on each of them, as follows:

```
nasm
global _start

section .text
_start:
    xor rax, rax
    xor rbx, rbx
    inc rbx
    add rax, rbx
```

This code should perform the exact same operations, but now in a more efficient way. Let's assemble our code, and run it with `gdb`:

```
$ ./assembler.sh fib.s -g
gef➤ b _start
Breakpoint 1 at 0x401000
```

```

gef> r
...SNIP...

code:x86:64 -----
→ 0x401001 <_start+1>      xor    eax, eax
   0x401003 <_start+3>      xor    ebx, ebx

registers ----- $rax    : 0x0
$rbx    : 0x0

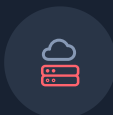
...SNIP...

code:x86:64 -----
→ 0x40100c                add    BYTE PTR [rax],
al-----
registers ----- $rax    : 0x1
$rbx    : 0x1

```

As we can see, `xor` ing our registers with themselves turned each of them to `0` 's, and the rest of the code performed the same operations as earlier, so we ended up with the same final values for both `rax` and `rbx` .

Connect to HTB



Pwnbox

Your own web-based Parrot Linux instance to play our labs.



Switching Pwnbox location will terminate the spawned Pwnbox.

Pwnbox Location

UK



Start Pwnbox

∞ spawns left

• Offline



Enable step-by-step solutions ⓘ

PRO

⚠ Answer questions in English to ensure an accurate response.

Question 1

 +1

 +40



Add an instruction to the end of the attached code to "xor" "rbx" with "15". What is the hex value of 'rbx' at the end?

Write your answer

 Submit

 Download File

 Previous

Section 12 / 24

 +20

Next

Mark Complete & Next